

Кратко в предыдущих сериях

Что советуем перед семинаром освежить в памяти из курса статистики:

- Описательные статистики: среднее, частота абсолютная и относительная, их поиск с помощью пакета `numpy`
- Генеральная совокупность, выборка
- Случайные величины, независимость случайных величин, их распределение
- Метод Монте-Карло
- Матожидание и дисперсия
- Точечные и интервальные оценки
- Оценки параметров среднего и пропорции

Тема 1. Интро.

0. Цель и задачи

Цель — научиться не попадаться на статистические заблуждения и решать некоторые задачи с собеседований.

Задачи:

- вспомнить статистику (пункты выше)
- познакомиться со статистическими парадоксами
- научиться моделировать некоторые из них
- научиться видеть подвох на данных и не делать некорректные выводы

1. Предисловие

1.1 Ценность этого семинара

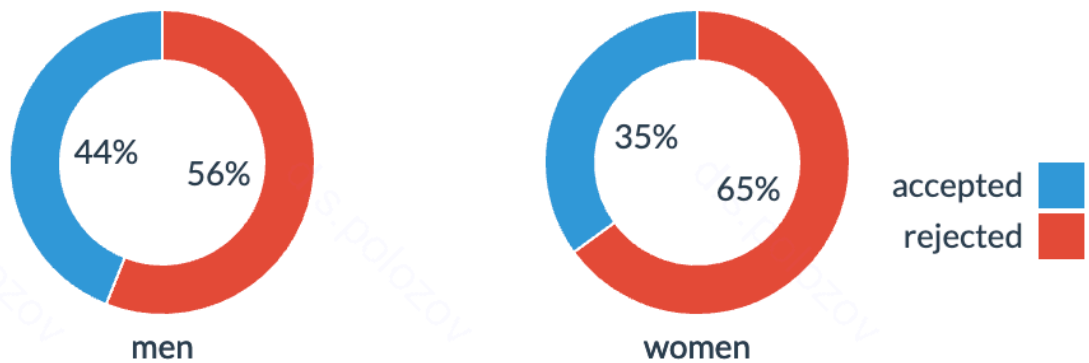
Ценность этого семинара в том, чтобы показать, что аналитика — это не просто подсчет циферок, изображение их на графиках и тривиальные выводы. Это нечто более сложное и творческое. Не всегда выводы, которые кажутся очевидно правдивыми, таковыми являются. Мы рассмотрим несколько примеров и в этом убедимся.

2. Статистические парадоксы

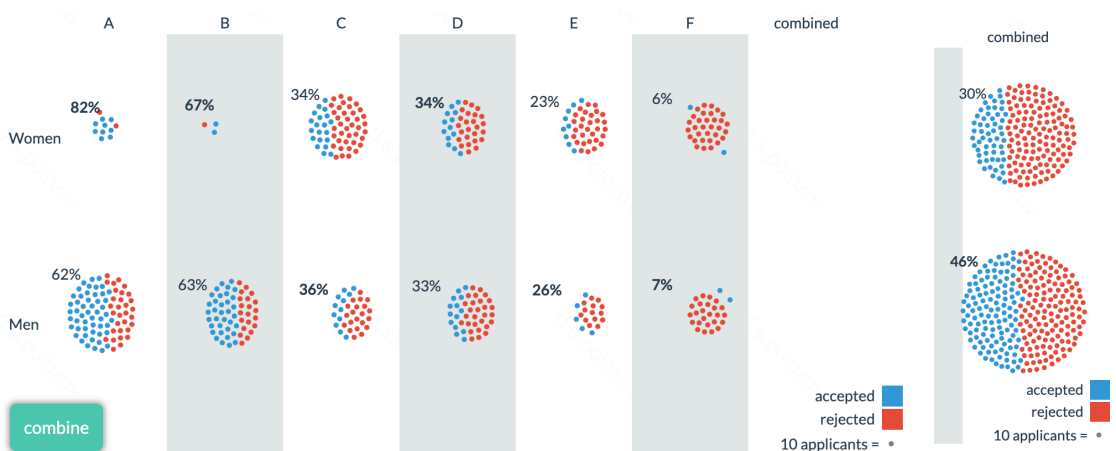
2.1. Парадокс Симпсона

На круговой диаграмме ниже представлены проценты прохождения в Калифорнийский университет Berkeley при поступлении в 1976 году. В связи с этим в на Berkeley был подан судебный иск за половую дискриминацию.

Что мы видим? — девушки значительно реже проходят отбор в школу, чем юноши: 30% против 46%.



Что же на самом деле происходит? — А дело все в разной демографии на разных факультетах!



Если присмотреться, то нестатзначимо больше берут юношей только на факультеты **C**, **E** и **F**, при этом девушек больше берут на факультеты **A**, **B** и **D**. А на факультете **A** и вовсе обратная дискриминация: девушек берут больше на 20%! Справедливости ради, на факультет **A** проходило отбор всего 108 девушек, однако разница 20% все равно впечатляет.

Вывод: итак, парадокс Симпсона заключается в том, что мы можем видеть в каждой из групп данных определенную зависимость, но при объединении этих групп зависимость исчезает или становится противоположной. Это происходит из-за того, что зависимость заключается в каких-то других факторах. Например, в примере со школами дискриминации нет ни по одному из полов, а девушки реже проходят отбор, потому что чаще пытаются пройти на более сложный факультет.

Продемонстрируем парадокс в действии: упростим ситуацию до 2 факультетов: "легкий" и "сложный". Пусть девушки больше стремятся пройти на сложный факультет, но при этом юноши берут и на легкий, и на сложный. Парадокс Симпсона проявляется в том, что при рассмотрении каждого факультета отдельно, девушки имеют более высокие показатели принятия, чем юноши, но при объединении данных, ситуация меняется.

```
Ввод [1]: 1 import numpy as np
2 import scipy.stats as sps
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 from tqdm.notebook import tqdm
6
7 import seaborn as sns
8 sns.set(font_scale=1, palette='Set2')
9 sns.set_style('whitegrid')
```

```
Ввод [2]: 1 simpson_df = pd.DataFrame(data={
2     'gender': ['male', 'female'],
3     'applied_cnt_easy': [2_000, 500],
4     'admitted_conversion_easy': [0.7, 0.8],
5     'applied_cnt_hard': [500, 2_000],
6     'admitted_conversion_hard': [0.4, 0.5],
7     'applied_cnt_overall': [2_500, 2_500],
8     'admitted_conversion_overall': [1_600 / 2_500, 1_400 / 2_500]
9 })
10 s = simpson_df.style
11
12 s.set_table_styles([ # create internal CSS classes
13     {'selector': '.true', 'props': 'background-color: #e6ffe6;'},
14     {'selector': '.false', 'props': 'background-color: #ffe6e6;'}],
15 overwrite=False)
16 cell_color = pd.DataFrame([['false', 'false', 'true'],
17                             ['true', 'true', 'false']],
18                             index=simpson_df.index,
19                             columns=simpson_df.columns[[2, 4, 6]])
20 s.set_td_classes(cell_color)
21 s
```

Out [2]:

	gender	applied_cnt_easy	admitted_conversion_easy	applied_cnt_hard	admitted_conversion_
0	male	2000	0.700000	500	0.40
1	female	500	0.800000	2000	0.50

Визуализируем это в виде pie-диаграмм

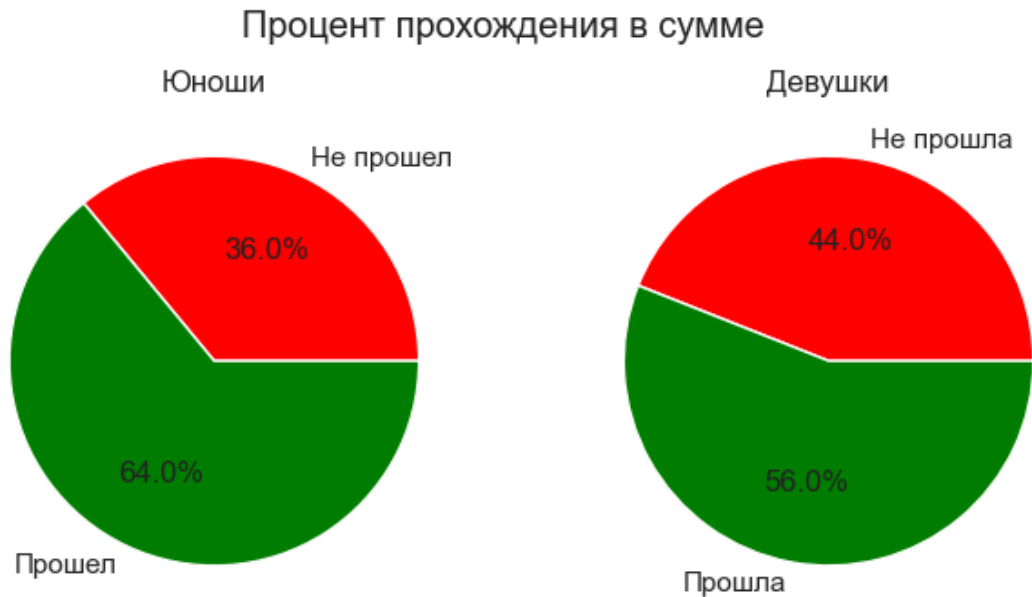
Ввод [3]:

```

1 fig, axes = plt.subplots(1, 4, figsize=(12, 3))
2 axes[0].pie([simpson_df.loc[0, 'applied_cnt_easy'] * (1 - simpson_
3             simpson_df.loc[0, 'applied_cnt_easy'] * simpson_df.loc[0,
4             colors = ['red', 'green'], labels=['Не прошел', 'Прошел'],
5 axes[0].set_title('Юноши, легкий')
6 axes[1].pie([simpson_df.loc[1, 'applied_cnt_easy'] * (1 - simpson_
7             simpson_df.loc[1, 'applied_cnt_easy'] * simpson_df.loc[1,
8             colors = ['red', 'green'], labels=['Не прошла', 'Прошла'],
9 axes[1].set_title('Девушки, легкий')
10
11 axes[2].pie([simpson_df.loc[0, 'applied_cnt_hard'] * (1 - simpson_
12             simpson_df.loc[0, 'applied_cnt_hard'] * simpson_df.loc[0,
13             colors = ['red', 'green'], labels=['Не прошел', 'Прошел'],
14 axes[2].set_title('Юноши, сложный')
15 axes[3].pie([simpson_df.loc[1, 'applied_cnt_hard'] * (1 - simpson_
16             simpson_df.loc[1, 'applied_cnt_hard'] * simpson_df.loc[1,
17             colors = ['red', 'green'], labels=['Не прошла', 'Прошла'],
18 axes[3].set_title('Девушки, сложный')
19 plt.suptitle('Процент прохождения по факультетам')
20 plt.show()
21
22 fig, axes = plt.subplots(1, 2, figsize=(8, 4))
23 axes[0].pie(
24     [simpson_df.loc[0, 'applied_cnt_overall'] * (1 - simpson_df.loc[0,
25     simpson_df.loc[0, 'applied_cnt_overall'] * simpson_df.loc[0,
26     colors = ['red', 'green'], labels=['Не прошел', 'Прошел'], ax
27 axes[0].set_title('Юноши')
28 axes[1].pie(
29     [simpson_df.loc[1, 'applied_cnt_overall'] * (1 - simpson_df.loc[1,
30     simpson_df.loc[1, 'applied_cnt_overall'] * simpson_df.loc[1,
31     colors = ['red', 'green'], labels=['Не прошла', 'Прошла'], ax
32 axes[1].set_title('Девушки')
33 plt.suptitle('Процент прохождения в сумме')
34 plt.show()

```





2.2. Парадокс Монти-Холла

Представьте, что вы стали участником игры, в которой вам нужно выбрать одну из трёх дверей. За одной из дверей находится автомобиль, за двумя другими дверями — козы. Вы выбираете одну из дверей, например, номер 1, после этого ведущий, который знает, где находится автомобиль, а где — козы, открывает одну из оставшихся дверей, например, номер 3, за которой находится коза. После этого он спрашивает вас — не желаете ли вы изменить свой выбор и выбрать дверь номер 2? Примете ли вы предложение ведущего или не измените свой выбор?

На первый взгляд кажется, что без разницы, менять дверь или нет: двери осталось 2, значит, за каждой из дверей вероятность встретить автомобиль 50%.

Вы уже знакомы с методом Монте-Карло, так давайте просимулируем игру на python! Пусть автомобиль находится за случайной дверью и игрок тоже выбирает случайную дверь. Сначала протестируем стратегию без смены двери, а потом — со сменой и сравним результаты.

```

Ввод [4]: 1 def monty_hall_game(change=False, n_iters=10_000):
2     results = []
3     doors = [1, 2, 3]
4     for _ in range(n_iters):
5         door_auto = np.random.choice(doors)
6         door_player = np.random.choice(doors)
7         doors_may_open = [door for door in doors if (door != door_auto)]
8         door_opened = np.random.choice(doors_may_open)
9         if change:
10            new_door_player = [door for door in doors if (door != door_opened)]
11            door_player = new_door_player
12
13
14            results.append(int(door_player == door_auto))
15
16    return results
17
18 results_change = monty_hall_game(change=True)
19 results_not_change = monty_hall_game(change=False)
20
21 print(f'результат, когда меняем дверь: {np.round(100 * np.mean(results_change), 1)}%')
22 print(f'результат, когда не меняем дверь: {np.round(100 * np.mean(results_not_change), 1)}%')

```

результат, когда меняем дверь: 66.2%

результат, когда не меняем дверь: 32.4%

Поразительно! В 2/3 случаев, когда мы меняем дверь, мы выигрываем, тогда как если не меняем, выигрываем только в 1/3 случаев.

Но как так? — Дело в том, что если не менять дверь, то вы выиграете с вероятностью, равной первоначальной, то есть, 1/3. При этом у вас есть 2 двери и 2 решения: менять дверь или нет, и где-то находится автомобиль (так как открыли дверь с козой). Значит, если вероятность выиграть без смены двери 1/3, то вероятность со сменой двери — 2/3!

Но почему 2/3? — Потому что исключение одной двери перенесло всю вероятность авто в ту, на которую вы можете поменять. Более формально — можно покопаться в условных вероятностях и получить требуемые 1/3 и 2/3 вероятности.

2.3. Ошибка выжившего

В одну американскую больницу направляли всех пострадавших в результате несчастных случаев в городе. Но больше всего в больницу попадало водителей и пассажиров, пострадавших в ДТП. Чтобы уменьшить их количество, городские власти сделали обязательным пользование ремнями безопасности. Водители и пассажиры стали пристегиваться ремнями. После этого число ДТП осталось неизменным, но вот число пострадавших в них людей, которые поступали в больницу, даже увеличилось. Вопрос: Почему?

На первый взгляд зависимость такая: начали пристегиваться → стало больше пострадавших → ремни неэффективны

Но что произошло на самом деле?

Пользование ремнями безопасности уменьшило число погибающих при ДТП. Многие люди, которые без ремня безопасности погибли бы и попали в морг, остались в живых, но были с травмами. Им требовалось лечение, а не вскрытие, поэтому число попадающих в больницу увеличивалось.

Это классическая **ошибка выжившего**. Эта ловушка мышления проявляется, когда мы смотрим только на результаты тех, кто дожил до успеха, и игнорируем невидимые неудачи. В данном случае мы не учли статистику по смертям, поэтому могли сделать неправильный вывод о полезности ремней безопасности.

2.4. Парадокс полицейского

2.4.1. Пререквезит: условная вероятность

- **Вероятность события A , если известно, что произошло событие B :**

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, P(B) > 0.$$

Пример: вероятность, что выпадет 6, если известно, что выпало четное число.

- **Независимость событий:** события A и B **независимы**, если $P(A \cap B) = P(A) \cdot P(B)$.

Пример: вероятность, что выпадет 6, если известно, что в прошлый раз выпало 3.

Замечание: независимость еще можно определить как $P(A|B) = P(A)$, если $P(B) \neq 0$.

- **Задача на условные вероятности:**

Задача: Игральные кости

Бросают две игральные кости. Известно, что сумма выпавших очков равна 8.

Вопрос:

- Какова вероятность того, что на первой кости выпало число 5?

Решение:

1. Пространство элементарных событий:

Всего возможных пар исходов при броске двух костей — $6 \times 6 = 36$.

2. Условие:

Рассматриваем только те исходы, где сумма очков равна 8:

$(2, 6), (3, 5), (4, 4), (5, 3), (6, 2)$.

Таким образом, всего таких исходов 5.

3. Интересующее событие:

На первой кости выпало 5. Подходящий исход — $(5, 3)$.

4. Условная вероятность:

По формуле условной вероятности:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}.$$

Вероятность события B (сумма равна 8):

$$P(B) = \frac{\text{Число подходящих исходов}}{\text{Общее число исходов}} = \frac{5}{36}.$$

Вероятность события $A \cap B$ (первая кость — 5, сумма — 8):

$$P(A \cap B) = \frac{1}{36}.$$

Условная вероятность:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{\frac{1}{36}}{\frac{5}{36}} = \frac{1}{5}.$$

Ответ:

Вероятность того, что на первой кости выпало число 5, если известно, что сумма равна 8, равна $\frac{1}{5}$.

2.4.2. Пререквизит: формула Байеса и формула полной вероятности

Теорема: (формула Байеса) $P(A|B) = \frac{P(B|A)P(A)}{P(B)}.$

Доказательство: по определению условной вероятности:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{P(B|A)P(A)}{P(B)}.$$

Теорема: (формула полной вероятности) если $A = \sqcup A_i$, то есть, A состоит из объединения непересекающихся событий A_i , то $P(B) = \sum P(B|A_i)P(A_i).$

Доказательство: так как A состоит из объединения непересекающихся событий A_i , то $B = \sum P(B \cap A_i)$, а каждое $P(B \cap A_i) = P(B|A_i)P(A_i)$ по определению условной вероятности.

Следствие: $P(A_j|B) = \frac{P(B|A_j)P(A_j)}{\sum P(B|A_i)P(A_i)}.$

2.4.3. Непосредственно, парадокс полицейского

- Алкотестер почти идеален:
 - Точность при наличии алкоголя: 100% (если человек пьян, алкотестер это определит всегда)
 - Точность при отсутствии алкоголя: 98% (если человек не пьян, алкотестер в 2% случаев скажет, что он пьян)
- Реально пьян только каждый тысячный человек. Полицейский проверил случайного человека и алкотестер показал, что он пьян. Полицейский сказал, что этот человек пьян с вероятностью 98% и взял штраф с человека.

Вопрос: прав ли полицейский?

Решение: распишем задачу с точки зрения статистики:

- пусть A — это событие "человек пьян"
- пусть B — это событие "алкотестер показал положительный результат"
- тогда нас интересует $P(A|B)$, т.е. вероятность, что человек пьян, при условии того, что алкотестер показал положительный результат
- $P(A) = 0.001$, так как по условию пьян каждый тысячный человек
- $P(B|A) = 1$, так как по условию алкотестер всегда показывает положительный результат, если человек пьян
- $P(B|\bar{A}) = 0.02$, так как по условию алкотестер показывает ложноположительный результат в 2% случаев
- распишем $P(A|B)$ по формуле Байеса:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B|A)P(A) + P(B|\bar{A})P(\bar{A})} = \frac{0.001}{0.001 + 0.02 \cdot 0.999} < 5\%.$$

Итого: если о подозреваемом ничего более неизвестно, то в таком случае положительные показания алкотестера даже близко не дают гарантии, что человек действительно пьян.

Вывод: полицейский совершает **ошибку базовой вероятности**. Он интерпретирует вероятность отрицательного результата при трезвости (98%) как вероятность пьяности при положительном результате алкотестера. Нужно уметь различать разные вероятности и правильно их интерпретировать.

2.5. Парадокс друзей

Почему твои друзья популярнее, чем ты (если ты не Кайли Дженнер)?

Согласно данным ученых из университета Южной Калифорнии, более,

Ввод [29]:

```
1 from graphviz import Graph as GVGraph
2 from itertools import permutations
```

```
Ввод [140]: 1 np.random.seed(seed=42)
2
3 persons = ['Алиса', 'Борис', 'Виталий', 'Геннадий', 'Елена', 'Жанна']
4 probs = dict(zip(persons, sps.beta(a=0.8, b=0.8).rvs(size=len(persons))))
5
6 relations = []
7 for pair in permutations(persons, 2):
8     if pair[0] < pair[1]:
9         friendship = sps.bernoulli((probs[pair[0]] + probs[pair[1]])/2)
10        if friendship == 1:
11            relations.append(pair)
12
13 relations
```

```
Out [140]: [('Алиса', 'Геннадий'),
('Алиса', 'Елена'),
('Алиса', 'Зиновий'),
('Алиса', 'Илья'),
('Борис', 'Геннадий'),
('Борис', 'Зиновий'),
('Борис', 'Илья'),
('Виталий', 'Геннадий'),
('Геннадий', 'Елена'),
('Геннадий', 'Жанна'),
('Геннадий', 'Зиновий'),
('Елена', 'Илья'),
('Жанна', 'Илья'),
('Зиновий', 'Илья')]
```


Ввод [141]:

```

1 class Vertex:
2     def __init__(self, name):
3         self.name = name
4         self.neighbors = set()
5
6     def add_neighbor(self, vertex):
7         self.neighbors.add(vertex)
8
9     def degree(self):
10        return len(self.neighbors)
11
12    def get_neighbors(self):
13        return list(self.neighbors)
14
15    def get_neighbor_by_name(self, name):
16        for neighbor in self.neighbors:
17            if neighbor.name == name:
18                return neighbor
19        return None
20
21    def __repr__(self):
22        return f"Vertex({self.name})"
23
24
25 class Graph:
26     def __init__(self):
27         self.vertices = {}
28
29     @classmethod
30     def from_edge_list(cls, edge_list):
31         graph = cls()
32         for from_v, to_v in edge_list:
33             graph.add_edge(from_v, to_v)
34         return graph
35
36     def add_vertex(self, name):
37         if name not in self.vertices:
38             self.vertices[name] = Vertex(name)
39         return self.vertices[name]
40
41     def add_edge(self, from_name, to_name):
42         from_vertex = self.add_vertex(from_name)
43         to_vertex = self.add_vertex(to_name)
44         from_vertex.add_neighbor(to_vertex)
45         to_vertex.add_neighbor(from_vertex)
46
47     def get_vertex(self, name):
48         """
49         Возвращает вершину по имени, либо None, если такой нет.
50         """
51         return self.vertices.get(name, None)
52
53     def visualize(self):
54         dot = GVGraph()
55         added_edges = set()
56         for vertex in self.vertices.values():
57             dot.node(vertex.name)
58             for neighbor in vertex.neighbors:
59                 edge = tuple(sorted((vertex.name, neighbor.name)))
60                 if edge not in added_edges:
61                     dot.edge(vertex.name, neighbor.name)

```

```

62         added_edges.add(edge)
63     return dot
64
65     def __repr__(self):
66         return f"Graph(vertices={list(self.vertices.keys())})"

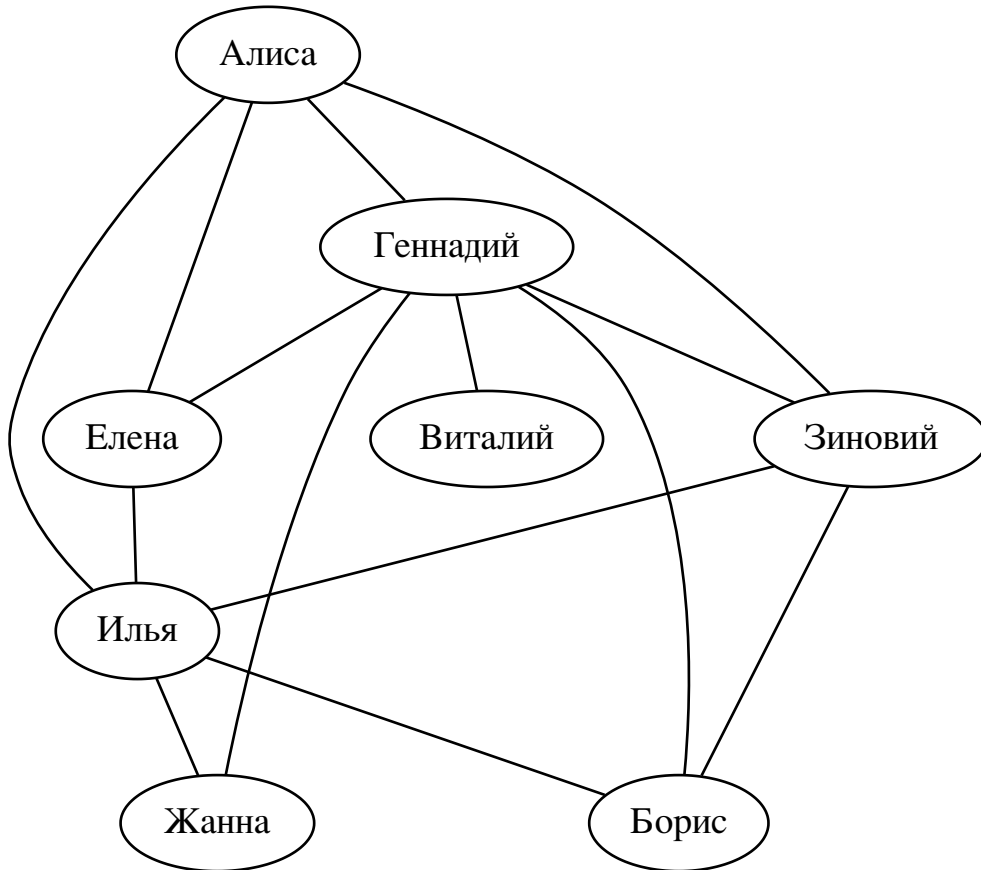
```

```

Ввод [142]: 1 friends_graph = Graph.from_edge_list(relations)
            2 friends_graph.visualize()

```

Out [142]:



```

Ввод [143]: 1 friends_graph.vertices

```

```

Out [143]: {'Алиса': Vertex(Алиса),
            'Геннадий': Vertex(Геннадий),
            'Елена': Vertex(Елена),
            'Зиновий': Vertex(Зиновий),
            'Илья': Vertex(Илья),
            'Борис': Vertex(Борис),
            'Виталий': Vertex(Виталий),
            'Жанна': Vertex(Жанна)}

```

```

Ввод [144]: 1 cnt_friends_df = pd.DataFrame(columns=['имя', 'кол-во друзей', 'ср
2
3 for person in persons:
4     vertex_person = friends_graph.get_vertex(person)
5     cnt_person = vertex_person.degree()
6     neighbours = vertex_person.get_neighbors()
7     cnt_friends_of_friend = []
8     for neighbour in neighbours:
9         cnt_friends_of_friend.append(neighbour.degree())
10
11     avg_cnt_friends_of_friend = np.mean(cnt_friends_of_friend)
12     cnt_friends_row = pd.DataFrame(data={'имя': [person],
13                                         'кол-во друзей': [cnt_per
14                                         'среднее кол-во друзей у
15     cnt_friends_df = pd.concat([cnt_friends_df, cnt_friends_row],
16
17 cnt_friends_df

```

Out [144]:

	имя	кол-во друзей	среднее кол-во друзей у друзей
0	Алиса	4	4.500000
1	Борис	3	5.000000
2	Виталий	1	6.000000
3	Геннадий	6	2.833333
4	Елена	3	5.000000
5	Жанна	2	5.500000
6	Зиновий	4	4.500000
7	Илья	5	3.200000

```

Ввод [147]: 1 cnt = len(cnt_friends_df[cnt_friends_df['кол-во друзей'] > cnt_fr
2
3 print(f'y {100 * np.round(1 - cnt / len(persons), 2)}% друзей мен

```

у 75.0% друзей меньше, чем у среднего их друга

Разберем пример попроще: пусть в школе есть 2 класса: в одном 100 человек, а во втором — 10 человек. У каждого ученика в школе спросили, сколько человек учится в его классе, а потом усреднили. Получилось $(100 \cdot 100 + 10 \cdot 10) / (100 + 10) \approx 92$ человека, хотя на самом деле среднее число учеников в классе $(100 + 10) / 2 = 55$. Смещение образовалось из-за того, что чаще спрашивали тех, кто учится в большем классе (так как их просто больше).

Вывод: Парадокс дружбы — это пример взвешенного среднего, где вес — это число связей у человека. Люди с большим числом связей оказывают непропорционально большое влияние на статистику, поэтому смещают ее в большую сторону.

Где еще можно встретиться с парадоксом дружбы:

- *парадокс ожидания поезда:* если спросить у людей, стоящих на платформе, сколько они ждут поезд, то получится в среднем больше, чем если оценить среднее время ожидания случайного человека

- *нагрузки на сервер*: если пользователи распределены неравномерно между серверами, то средний пользователь будет себя чувствовать в среднем хуже, чем средний по всем серверам
- *предупреждение эпидемий*: при предупреждении эпидемий обычно смотрят на здоровье не случайных людей, а друзей случайных людей. Все потому, что друзья имеют в среднем больше контактов, а значит, лучше в первую очередь оградить их при наличии у них симптомов

3. Итоги семинара

С чем выходим с занятия:

- разобрались, почему не стоит делать выводы на средних, если мы не знаем, случайно ли были собраны группы (парадокс Симпсона)
- увидели, что стоит пересчитывать вероятность событий, опираясь на новую информацию (не путать с подглядыванием: если подгляднуть, то можно преждевременно открыть дверь с козой, даже не дождавшись информации от ведущего)
- разобрались, почему не стоит делать выводы на смещенной выборке (ошибка выжившего)
- увидели, что нужно глубоко понимать разные вероятности и правильно их интерпретировать
- увидели на примере парадокс дружбы, повысили себе самооценку и поняли, как эту информацию можно использовать на практике

4. Полезные материалы

- [Иллюстрация с парадоксом Симпсона \(https://setosa.io/simpsons/\)](https://setosa.io/simpsons/)
- [Шортс про парадокс выжившего \(https://www.youtube.com/shorts/bFg1TCLRk7s\)](https://www.youtube.com/shorts/bFg1TCLRk7s)
- [Про условную вероятность \(там еще в ссылках формула Байеса и формула полной вероятности\)](https://ru.wikipedia.org/wiki/%D0%A3%D1%81%D0%BB%D0%BE%D0%B2%D0%BF)
(https://ru.wikipedia.org/wiki/%D0%A3%D1%81%D0%BB%D0%BE%D0%B2%D0%BF)
- [Пардокс полицейского или ошибка базового процента на вики](https://ru.wikipedia.org/wiki/%D0%9E%D1%88%D0%B8%D0%B1%D0%BA%D0%B0)
(https://ru.wikipedia.org/wiki/%D0%9E%D1%88%D0%B8%D0%B1%D0%BA%D0%B0)
- [Статья на хабре про парадокс дружбы \(https://habr.com/ru/articles/383787/\)](https://habr.com/ru/articles/383787/)
- [Статья про парадокс дружбы](https://web.archive.org/web/20141019095712/http://www-scf.usc.edu/~kooti/files/FriendParadoxRedux.pdf)
(https://web.archive.org/web/20141019095712/http://www-scf.usc.edu/~kooti/files/FriendParadoxRedux.pdf)